



Chapter 4

Obtaining the Correct Output Presentation

IN THIS CHAPTER

» Defining a correct output presentation

» Considering the needs of the audience

» Determining which graph type to use

» Interacting with external data

.....

Throughout this book, you discover methods for obtaining, cleaning, and analyzing data using a wide variety of techniques. Oddly enough, data scientists spend most of their time cleaning the data (see

<https://www.analyticsindiamag.com/6-tasks-data-scientists-spend-the-most-time-doing/> for details). The next most common task is simply obtaining the data. Yet, all this effort doesn't amount to anything if no one can understand the results. Other data scientists are likely to understand, but you must work with people who don't have six degrees in math, two in computer science, and another in logic. Consequently, simply generating output that depicts the results of your efforts isn't enough; you must generate output that speaks to the viewer, which is the first consideration in this chapter.

In some respects, you become a detective when it comes to output, because what you really need is a profile of the people who will look at the output you create. If you don't understand the needs of the people who view your analysis, you won't ever create useful output for them. The problem is that people differ widely around the world with regard to what they expect given their occupation, environment, and cultural norms, among many other factors. So, no boxed solution exists to address

the problem of how to present your output in a manner that others will understand. You must prepare a profile, which is the purpose of the second part of this chapter.

Just like everyone else, data scientists get stuck in a rut. If you view examples online, you find that most of them use scatterplots, followed by line graphs. In some cases, you find rare examples of histograms or boxplots, depending on the skill of the data scientist. Fortunately, you have many other options at your disposal, and the third section of this chapter explores only a few of them.

Oddly enough, in a world filled with emoji and many other assorted graphic presentations, the data analyses provided in most of the examples you see online are devoid of any sort of decoration. Most of them are outright bland to the point of making you somnolent. The last section of this chapter addresses the use of external data, such as graphics, to add a bit of pizzazz to your presentation.



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0604_Presentation.ipynb` source code file for Python and the `DSPD_R_0604_Presentation.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Considering the Meaning of Correct

The term *correct* is a loaded one. In fact, most terms are loaded because each could mean something different to every person reading this text. What is correct to one person can be incorrect to another based on a wealth of qualifiers, such as:

- Bias
- Experience
- Environment
- Personal knowledge
- Technological changes
- World events

In fact, it wouldn't take long to compile a long enough list of qualifiers to make any discussion of correctness meaningless. However, at some point you must decide that a particular output presentation is correct. The next section of this chapter discusses what the presentation must contain to be correct. However, the presentation alone doesn't determine correctness overall.



REMEMBER A good starting point for ensuring that the output of your analysis contains the fewest possible mistruths is to use the material from the “[Considering the Five Mistruths in Data](#)” section of [Chapter 2](#) of this mini-book. The fact that your output is believable will make it more correct as well. However, the essential characteristic of data you have vetted and manicured for the highest level of truthfulness is that you also believe in that data, which in turn makes you, as a person, more believable.

The fit and finish of your output also matters. Ensuring that you use the correct kind of graphic output is essential because some types of graphic output can actually obscure the point you want to make, thereby making it less correct. Adding various kinds of graphics and other external data is important because it helps you draw a parallel between the abstract data in your presentation and the physical world.

The issue of correctness is also personal. Always remember that the correctness you perceive may appear as incorrectness to someone else, so getting a second, third, or fourth opinion is helpful, too. Of course, presentation by committee leads to the muddying of your presentation, also making it less correct. The clarity of the vision of an individual augmented by insights provided by a small number of others generally results in a presentation that is correct from a number of perspectives, but remains focused.

Determining a Presentation Type

The presentation of your analysis output is how it appears to others. Of course, how they see it depends on their point of view. If you ask any number of people to look at a painting of nearly anything in nearly any style, each person will likely see something slightly different. The painting doesn't change; the perspective of the person does. Consequently, the presentation of the output of an analysis matters and is one of the deter-

mining factors in conveying information to others in an unambiguous manner.

Considering the audience

One of the most difficult and confounding aspects of creating a presentation is frame of reference. Your audience has

- Had certain experiences
- Obtained a particular level of education
- Engaged in a particular set of jobs
- A need for specific kinds of information

The inability for various groups to communicate stems from the lack of a frame of reference between them. Part of your task is to try to see things from the perspective of your audience, which may feel uncomfortable but is a requirement if you want to communicate your ideas and the results of your analysis.



TIP

Avoid using stereotypes to define your audience. Even seasoned presenters fall flat when sizing up an audience incorrectly. One audience may have certain characteristics in common with other audiences, but each audience is unique in one or more ways. It's important to consider these differences whether your method of presentation is in person, through writing, or by various means online.

Defining a depth of detail

As a data scientist, you may find that you like to see all the details. In fact, you might be downright exuberant about details and find them fascinating. Not every audience wants details, however. You may have encountered a manager who felt that a presentation was mired in unnecessary detail. All the manager needed was an overview. The manager hired others to deal with the details, but needed just an overview to get the big picture as part of running a large business. If your audience doesn't need detailed information, that fact affects everything from the amount of data you provide to the kind of graphics you choose.

Another issue to consider is that when an audience does need detail, the details may reflect specific needs. The audience might not need to know

about the raw data but may instead need only to look at specific areas of the manicured data you provide. In fact, you may find that you have to research particular areas to provide more detail than needed to perform your analysis. A particular presentation may require that you provide historical information with the data. You don't need the historical information to perform the analysis, but your audience needs it to make sense of the data.

**TIP**

When considering the depth of detail for a presentation, look at other successful reports, videos, or oral presentations for the same group. You know when a source is successful because people talk about it, and you see it cited in still other reports. Look for the buzz factor when thinking about depth of detail because anyone can get friends to provide positive reviews, but no one can generate buzz as an act of will.

Ensuring that the data is consistent with audience needs

The attention span of your audience is likely short. In fact, according to the infographic at

<https://www.digitalinformationworld.com/2018/09/the-human-attention-span-infographic.html>, humans now have a shorter attention span than goldfish. Consequently, you may have as little as nine seconds before your audience assumes a blank expression and enters its own little world, which doesn't include you or your data. To keep your audience from becoming comatose, you need to provide data that the audience wants and needs:

- **Wants:** Has a strong desire to obtain, even when the data isn't needed, usually because the information has become popular. People's wants change so fast that you may get dizzy tracking them.
- **Needs:** Required to perform tasks or to discover something new related to an occupation or activity. The audience may not recognize a need, especially when a need isn't popular. You know when you've been successful because the audience will suddenly experience that lightbulb moment when the need becomes clear. Needs tend to change slowly.

**TIP**

You likely used an immense database to perform your analysis, and if you present all of it, your audience will go from comatose to som-

nolent to fully asleep. By drawing up a list of wants and needs for this particular audience, you can whittle the data down to a manageable size. The important thing is to focus on just a few of these wants and needs based on what you know about your audience. Remember that you need to grab the audience's undivided attention in only nine seconds.

Understanding timeliness

The timeliness of your data depends on the sort of information you present and the focus of your audience. When performing a medical analysis for a disease with a long history, you might use data that's 10, 15, or even 20 years old, especially when the presentation is for medical researchers (versus family practitioners) who are looking for trends. Your data may span hundreds or thousands of years when talking with historians. However, if your analysis is for graphics designers who specialize in website design, having data any more than a year old may be ruinous. Deciding on how old is too old is an essential part of creating your presentation.

Choosing the Right Graph

The kind of graph you choose determines how people view the associated data, so choosing the right graph from the outset is important. For example, if you want to show how various data elements contribute toward a whole, you really need to use a pie chart. On the other hand, when you want people to form opinions on how data elements compare, you use a bar chart. The idea is to choose a graph that naturally leads people to draw the conclusion that you need them to draw about the data that you've carefully massaged from various data sources. The following sections describe the various graph types and offer basic examples of how to use them.

Telling a story with your graphs

You use the graphics you create to tell a story pictorially. The goal isn't to display data; you can use a table to do that. Instead, the goal is to see the data in a particular way that doesn't lend itself easily to either tables or text.



REMEMBER The difference between presenting data and telling a story is important because many people don't relate well to abstract information. You can show them the data, explore it in detail, and consider every possible approach to demonstrating that the data contains useful patterns, yet your audience still won't understand. As an example from this book, [Chapter 3](#) of this minibook shows how underfitting appears in [Figure 3-1](#). The graphic nature of the example shows in concrete terms what underfitting is all about and in a way that words really can't adequately describe.

When working with graphics, think about the story you want to tell using those graphics. A graph can have characters involved with it. You can show chases and escapes, deal a death blow to a nefarious villain, and show how the good person wins. If you can't see the story in your graphic, neither will your audience.

Showing parts of a whole with pie charts

Pie charts focus on showing parts of a whole. The entire pie represents 100 percent. The question is how much of that percentage each value occupies. The following example shows how to create a pie chart with many of the special features in place:

```
import matplotlib.pyplot as plt

%matplotlib inline

values = [5, 8, 9, 10, 4, 7]

colors = ['b', 'g', 'r', 'c', 'm', 'y']

labels = ['A', 'B', 'C', 'D', 'E', 'F']

explode = (0, 0.2, 0, 0, 0, 0)

plt.pie(values, colors=colors, labels=labels,
```

```
explode=explode, autopct='%1.1f%%',
```

```
counterclock=False, shadow=True)
```

```
plt.title('Values')
```

```
plt.show()
```

The essential part of a pie chart is the values. You could create a basic pie chart using just the values as input.

The `colors` parameter lets you choose custom colors for each pie wedge. You use the `labels` parameter to identify each wedge. In many cases, you need to make one wedge stand out from the others, so you add the `explode` parameter with a list of explode values. A value of 0 keeps the wedge in place; any other value moves the wedge out from the center of the pie.

Each pie wedge can show various kinds of information. This example shows the percentage occupied by each wedge with the `autopct` parameter. You must provide a format string to format the percentages.



TIP

Some parameters affect how the pie chart is drawn. Use the `counterclock` parameter to determine the direction of the wedges. The `shadow` parameter determines whether the pie appears with a shadow beneath it (for a 3-D effect). You can find other parameters at https://matplotlib.org/api/pyplot_api.html.

In most cases, you also want to give your pie chart a title so that others know what it represents. You do this using the `title()` function. [Figure 4-1](#) shows the output from this example.

Creating comparisons with bar charts

Bar charts make comparing values easy. The wide bars and segregated measurements emphasize the differences between values, rather than the flow of one value to another, as a line graph would do. Fortunately, you have all sorts of methods at your disposal for emphasizing specific

values and performing other tricks. The following example shows just some of the things you can do with a vertical bar chart:

```
import matplotlib.pyplot as plt

%matplotlib inline

values = [5, 8, 9, 10, 4, 7]

widths = [0.7, 0.8, 0.7, 0.7, 0.7, 0.7]

colors = ['b', 'r', 'b', 'b', 'b', 'b']

plt.bar(range(0, 6), values, width=widths,

        color=colors, align='center')

plt.show()
```

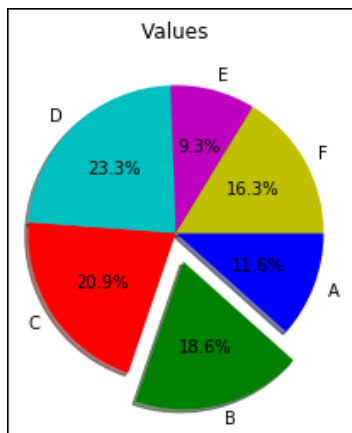


FIGURE 4-1: Pie charts show a percentage of the whole.

To create even a basic bar chart, you must provide a series of x coordinates and the heights of the bars. The example uses the `range()` function to create the x coordinates, and `values` contains the heights.

Of course, you may want more than a basic bar chart, and Matplotlib provides a number of ways to get the job done. In this case, the example uses the `width` parameter to control the width of each bar, emphasizing the second bar by making it slightly larger. The larger width would show

up even in a black-and-white printout. The example also uses the `color` parameter to change the color of the target bar to red. (The rest are blue.)

As with other chart types, the bar chart provides some special features that you can use to make your presentation stand out. The example uses the `align` parameter to center the data on the x coordinate. (The standard position is to the left.) You can also use other parameters, such as `hatch`, to enhance the visual appearance of your bar chart. [Figure 4-2](#) shows the output of this example.

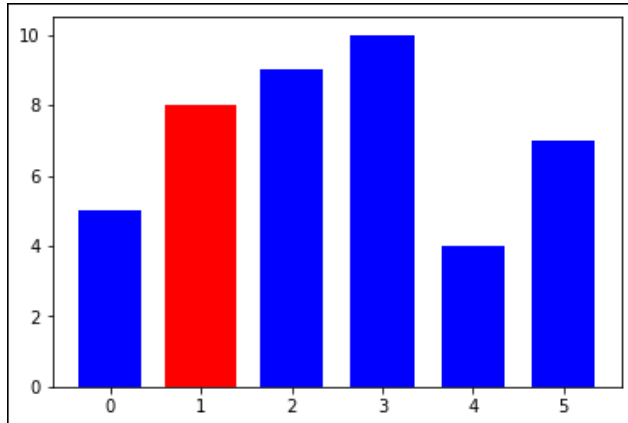


FIGURE 4-2: Bar charts make performing comparisons easier.



TIP

This chapter helps you get started using Matplotlib to create a variety of chart and graph types. Of course, more examples are better, so you can also find some more advanced examples on the Matplotlib site at <https://matplotlib.org/1.2.1/examples/index.html>. Some of the examples, such as those that demonstrate animation techniques, become quite advanced, but with practice you can use any of them to improve your own charts and graphs.

Showing distributions using histograms

Histograms categorize data by breaking it into *bins*, where each bin contains a subset of the data range. A histogram then displays the number of items in each bin so that you can see the distribution of data and the progression of data from bin to bin. In most cases, you see a curve of some type, such as a bell curve. The following example shows how to create a histogram with randomized data:

```
import numpy as np

import matplotlib.pyplot as plt

%matplotlib inline

x = 20 * np.random.randn(10000)

plt.hist(x, 25, range=(-50, 50), histtype='stepfilled',
         align='mid', color='g', label='Test Data')

plt.legend()

plt.title('Step Filled Histogram')

plt.show()
```

In this case, the input values are a series of random numbers. The distribution of these numbers should show a type of bell curve. As a minimum, you must provide a series of values, `x` in this case, to plot. The second argument contains the number of bins to use when creating the data intervals. The default value is 10. Using the `range` parameter helps you focus the histogram on the relevant data and exclude any outliers.

You can create multiple histogram types. The default setting creates a bar chart. You can also create a stacked bar chart, stepped graph, or filled stepped graph (the type shown in the example). In addition, you can control the orientation of the output, with vertical as the default.

As with most other charts and graphs in this chapter, you can add special features to the output. For example, the `align` parameter determines the alignment of each bar along the baseline. Use the `color` parameter to control the colors of the bars. The `label` parameter doesn't actually appear unless you also create a legend (as shown in this example). [Figure 4-3](#) shows typical output from this example.

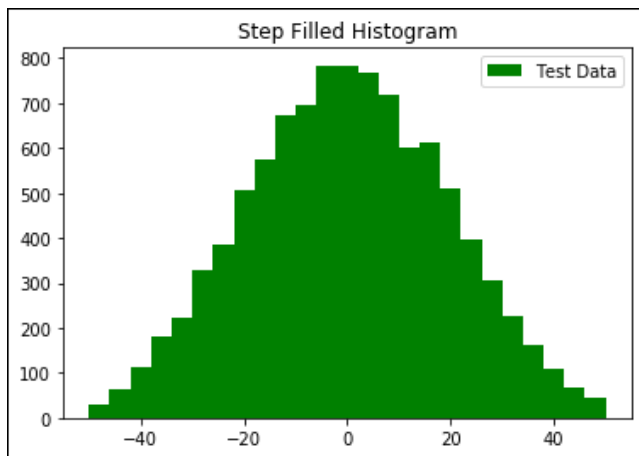


FIGURE 4-3: Histograms let you see distributions of numbers.



REMEMBER Random data varies call by call. Every time you run the example, you see slightly different results because the random-generation process differs.

Depicting groups using boxplots

Boxplots provide a means of depicting groups of numbers through their *quartiles* (three points dividing a group into four equal parts). A boxplot may also have lines, called *whiskers*, indicating data outside the upper and lower quartiles. The spacing shown within a boxplot helps indicate the skew and dispersion of the data. The following example shows how to create a boxplot with randomized data:

```
import numpy as np

import matplotlib.pyplot as plt

%matplotlib inline

spread = 100 * np.random.rand(100)

center = np.ones(50) * 50

flier_high = 100 * np.random.rand(10) + 100

flier_low = -100 * np.random.rand(10)
```

```
data = np.concatenate((spread, center,  
  
                        flier_high, flier_low))  
  
  
plt.boxplot(data, sym='gx', widths=.75, notch=True)  
  
plt.show()
```

To create a usable dataset, you need to combine several different number-generation techniques, as shown at the beginning of the example. Here's how these techniques work:

- `spread`: Contains a set of random numbers between 0 and 100
- `center`: Provides 50 values directly in the center of the range of 50
- `flier_high`: Simulates outliers between 100 and 200
- `flier_low`: Simulates outliers between 0 and -100

The code combines all these values into a single dataset using `concatenate()`. Being randomly generated with specific characteristics (such as a large number of points in the middle), the output will show specific characteristics but will work fine for the example.

The call to `boxplot()` requires only `data` as input. All other parameters have default settings. In this case, the code sets the presentation of outliers to green Xs by setting the `sym` parameter. You use `widths` to modify the size of the box (made extra large in this case to make the box easier to see). Finally, you can create a square box or a box with a notch using the `notch` parameter (which normally defaults to False). [Figure 4-4](#) shows typical output from this example.

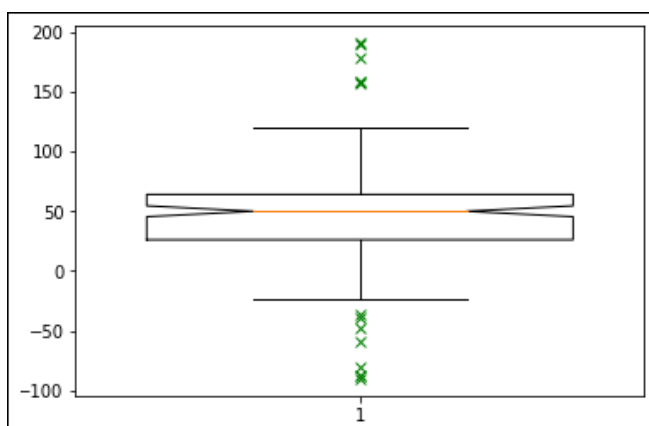


FIGURE 4-4: Use boxplots to present groups of numbers.

The box shows the three data points as the box, with the red line in the middle being the median. The two black horizontal lines connected to the box by whiskers show the upper and lower limits (for four quartiles). The outliers appear above and below the upper and lower limit lines as green Xs.

Defining a data flow using line graphs

One of the most commonly used graphs in this book is the line graph, which shows trends in data. You see not only the data points but also the flow between the data points. Using a line graph helps you to see the predictive nature of data science by drawing a line showing the missing data between the existing data points. What you receive is a continuous view of the data — at least, if the data is perfect. (See [Chapter 3](#) of this mini-book for a discussion of overfitting and underfitting of data.) The following example shows how to create a line graph with multiple lines:

```
import matplotlib.pyplot as plt

%matplotlib inline

x = range(0, 6)

y1 = [5, 8, 9, 10, 4, 7]
y2 = [2, 3, 0, 11, 5, 6]

plt.plot(x, y1, 'o--g', markersize=4, linewidth=1)
plt.plot(x, y2, 'h-.b', markersize=10,
         markeredgecolor='red',
         markerfacecolor='purple', markevery=2,
         dash_capstyle='round', linewidth=3)

plt.legend(['Old', 'New'])
```

```
plt.show()
```

This line graph uses a few features that you don't actually see in the book. The first line is standard, but with the addition of a specific marker size and line width. The second line includes all sorts of addition features that emphasize the New option over the Old option. The point of this example is that you don't have to stick with any particular set of rules. If you want your audience to pay particular attention to something, emphasize it.

[Figure 4-5](#) shows the output from this example.

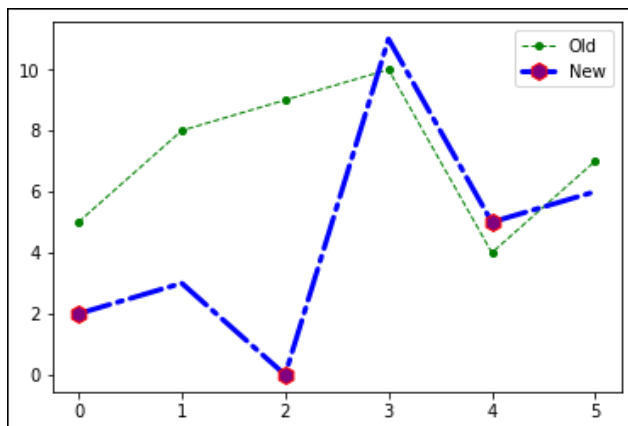


FIGURE 4-5: Use line graphs to show trends.

Seeing data patterns using scatterplots

Scatterplots show clusters of data rather than trends (as with line graphs) or discrete values (as with bar charts). The purpose of a scatterplot is to help you see data patterns. The following example shows how to create a scatterplot using randomized data:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
%matplotlib inline
```

```
x1 = 5 * np.random.rand(40)
```

```
x2 = 5 * np.random.rand(40) + 25
```

```
x3 = 25 * np.random.rand(20)
```

```
x = np.concatenate((x1, x2, x3))
```

```
y1 = 5 * np.random.rand(40)
```

```
y2 = 5 * np.random.rand(40) + 25
```

```
y3 = 25 * np.random.rand(20)
```

```
y = np.concatenate((y1, y2, y3))
```

```
plt.scatter(x, y, s=[100], marker='^', c='m')
```

```
plt.show()
```

The example begins by generating random x and y coordinates. For each x coordinate, you must have a corresponding y coordinate. Creating a scatterplot is possible using just the x and y coordinates.

You can dress up a scatterplot in a number of ways. In this case, the `s` parameter determines the size of each data point. The `marker` parameter determines the data point shape. You use the `c` parameter to define the colors for all the data points, or you can define a separate color for individual data points. [Figure 4-6](#) shows the output from this example.

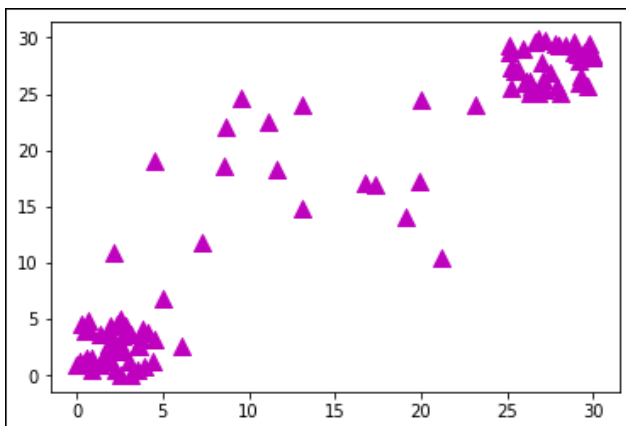


FIGURE 4-6: Use scatterplots to show groups of data points and their associated patterns.



REMEMBER You actually see a large number of scatterplots in this book because they appear quite commonly in data science output. Book 5, [Chapter 6](#) shows how to create some advanced scatterplot presentations that you find helpful with complex data. The point is that you shouldn't rely too much on scatterplots and line graphs. Other kinds of presentations may help your viewer focus on the data in ways that you hadn't originally envisioned.

Working with External Data

Pictures say a lot of things that words can't (or at least they do it with far less effort). Notebook is both a coding platform and a presentation platform. You may be surprised at just what you can do with it. The following sections provide a brief overview of some of the more interesting features.

Embedding plots and other images

At some point, you might have spotted a notebook with multimedia or graphics embedded into it and wondered why you didn't see the same effects in your own files. In fact, all the graphics examples in the book appear as part of the code. Fortunately, you can perform some more magic by using the `%matplotlib` magic function. The possible values for this function are: `'gtk'`, `'gtk3'`, `'inline'`, `'nbagg'`, `'osx'`, `'qt'`, `'qt4'`, `'qt5'`, `'tk'`, and `'wx'`, each of which defines a different *plotting backend* (the code used to actually render the plot) for presenting information onscreen.

When you run `%matplotlib inline`, any plots you create appear as part of the document. That's how [Figure 4-1](#), earlier in this chapter, shows the plot it creates immediately below the affected code.

Loading examples from online sites

Because some examples you see online can be hard to understand unless you have them loaded on your own system, you should also keep the `%load` magic function in mind. All you need is the URL of an example you want to see on your system. For example, try `%load` https://matplotlib.org/_downloads/pyplot_text.py. When you click

Run Cell, Notebook loads the example directly in the cell and comments the `%load` call out, as shown in [Figure 4-7](#). (The actual code is much longer than that shown in the figure.) You can then run the example and see the output from it on your own system.

Loading examples from online sites

```
In [ ]: # %load https://matplotlib.org/_downloads/pyplot_text.py
        """
        =====
        Pyplot Text
        =====

        """
        import numpy as np
        import matplotlib.pyplot as plt

        # Fixing random state for reproducibility
        np.random.seed(19680801)

        mu, sigma = 100, 15
        x = mu + sigma * np.random.randn(10000)
```

FIGURE 4-7: Load external code as needed to provide specific information for your presentation.

Obtaining online graphics and multimedia

A lot of the functionality required to perform special multimedia and graphics processing appears within `IPython.display`. By importing a required class, you can perform tasks such as embedding images into your notebook. Here's an example of embedding one of the pictures from the author's blog into the notebook for this chapter:

```
from IPython.display import Image, display

Embed = Image(

'http://blog.johnmuellerbooks.com/' +

'wp-content/uploads/2015/04/Layer-Hens.jpg',

width=600, height=450,

metadata={'Animal': 'Chicks'})

print('A picture of: ' + Embed.metadata['Animal'])

display(Embed)
```

The code begins by importing the required class, `Image`, and then using features from it to first define what to embed and then actually embed the image. Notice that you can attach metadata to the image and display it later. The output you see from this example appears in [Figure 4-8](#).



FIGURE 4-8: Embedding images can dress up your notebook presentation.



TIP

If you expect an image to change over time, you might want to create a link to it instead of embedding it. You must refresh a link because the content in the notebook is only a reference rather than the actual image. However, as the image changes, you see the change in your notebook as well. To accomplish this task, you use `SoftLinked = Image(url=' http://blog.johnmuellerbooks.com/wp-content/uploads/2015/04/Layer-Hens.jpg ')` instead of `Embed`.

When working with embedded images on a regular basis, you might want to set the form in which the images are embedded. For example, you may prefer to embed them as PDFs. To perform this task, you use code similar to this:

```
from IPython.display import set_matplotlib_formats
```

```
set_matplotlib_formats('pdf', 'svg')
```

You have access to a wide number of formats when working with a notebook. The commonly supported formats are 'png', 'retina', 'jpeg', 'svg', and 'pdf'.

The IPython display system is nothing short of amazing, and this section hasn't even begun to tap the surface for you. For example, you can import a YouTube video and place it directly into your notebook as part of your presentation if you want. You can see quite a few more of the display features demonstrated at

<http://nbviewer.jupyter.org/github/ipython/ipython/blob/1.x/examples/notebooks/Part%206%20Rich%20Display%20System.ipynb>.

Table of contents collapsed